

QUESTION 5

MINI LIBRARY SYSTEM

Kotlin OOP Assignment

NAME	Satara Chishti
ROLL NUMBER	2447
TEACHER	Prof. Arshad
DEPARTMENT	Computer Science
SUBJECT	Mobile App Development

Question 5 — Mini Library System

QUESTION 5 — REQUIREMENTS

Build a small library system.

Book: Create a Book data class containing title, author and price.

LibraryItem: Create an abstract class containing common information for library items. Create two subclasses: BookItem and MagazineItem.

Borrowable: Create interface Borrowable with borrow() and returnItem(). Make the appropriate classes implement it.

Library: Maintain a list of library items. Provide functions to add item, display items, borrow an item and return an item.

ItemStatus: Create an enum with AVAILABLE and BORROWED.

ID Generation: Use a companion object to generate IDs.

LibraryLogger: Create a singleton object and use it when an item is borrowed or returned.

Must use: Abstract class, inheritance, interface, polymorphism, data class, enum class, companion object, object, composition and encapsulation.

Deliverables: Book, LibraryItem, BookItem, MagazineItem, Borrowable, ItemStatus, Library, LibraryLogger, companion object, main(), at least 3 items, display, borrow, repeated borrow attempt, return, updated status, logger messages and polymorphism.

KOTLIN SOLUTION

```
// Question 5 – Mini Library System
data class Book(
    val title: String,
    val author: String,
    val price: Double
)

abstract class LibraryItem(
    val id: Int,
    val title: String
) {
    var status: ItemStatus = ItemStatus.AVAILABLE
    private set
    protected fun setStatus(newStatus: ItemStatus) {
        status = newStatus
    }
    abstract fun displayInfo()
    companion object {
        private var nextId = 1
        fun generateId(): Int = nextId++
    }
}

interface Borrowable {
    fun borrow()
    fun returnItem()
}

enum class ItemStatus {
    AVAILABLE,
    BORROWED
}

class BookItem(
    private val book: Book,
    id: Int
) : LibraryItem(id, book.title), Borrowable {
    override fun displayInfo() {
        println("Book #\$id | \${book.title} | \${book.author} | Rs.\${book.price} | \$status")
    }
    override fun borrow() {
        if (status == ItemStatus.AVAILABLE) {
            setStatus(ItemStatus.BORROWED)
            LibraryLogger.log("Book '\${book.title}' borrowed.")
        } else {
            LibraryLogger.log("Book '\${book.title}' is already borrowed.")
        }
    }
    override fun returnItem() {
        if (status == ItemStatus.BORROWED) {
            setStatus(ItemStatus.AVAILABLE)
            LibraryLogger.log("Book '\${book.title}' returned.")
        }
    }
}

class MagazineItem(
    private val name: String,
    private val issueNumber: Int,
    id: Int
) : LibraryItem(id, name), Borrowable {
    override fun displayInfo() {
        println("Magazine #\$id | \$name | Issue: \$issueNumber | \$status")
    }
    override fun borrow() {
        if (status == ItemStatus.AVAILABLE) {
            setStatus(ItemStatus.BORROWED)
            LibraryLogger.log("Magazine '\$name' borrowed.")
        } else {
            LibraryLogger.log("Magazine '\$name' is already borrowed.")
        }
    }
    override fun returnItem() {
        if (status == ItemStatus.BORROWED) {
            setStatus(ItemStatus.AVAILABLE)
            LibraryLogger.log("Magazine '\$name' returned.")
        }
    }
}

class Library {
```

```

    private val items = mutableListOf<LibraryItem>()
    fun addItem(item: LibraryItem) {
        items.add(item)
    }
    fun displayItems() {
        for (item in items) item.displayInfo()
    }
    fun borrowItem(id: Int) {
        items.find { it.id == id }?.borrow()
    }
    fun returnItem(id: Int) {
        items.find { it.id == id }?.returnItem()
    }
    fun getItems(): List<LibraryItem> = items
}

object LibraryLogger {
    fun log(message: String) {
        println("[LOG] $message")
    }
}

fun main() {
    val item1 = BookItem(
        Book("Kotlin Basics", "John Smith", 1200.0),
        LibraryItem.generateId()
    )
    val item2 = BookItem(
        Book("OOP Programming", "Sara Khan", 1500.0),
        LibraryItem.generateId()
    )
    val item3 = MagazineItem(
        "Tech Monthly", 25, LibraryItem.generateId()
    )
    val library = Library()
    library.addItem(item1)
    library.addItem(item2)
    library.addItem(item3)

    println("===== ALL LIBRARY ITEMS =====")
    library.displayItems()
    println("\n===== BORROW ITEM 1 =====")
    library.borrowItem(item1.id)
    println("\n===== BORROW ITEM 1 AGAIN =====")
    library.borrowItem(item1.id)
    println("\n===== RETURN ITEM 1 =====")
    library.returnItem(item1.id)
    println("\n===== UPDATED STATUS =====")
    library.displayItems()
    println("\n===== POLYMORPHISM =====")
    for (item: LibraryItem in library.getItems()) {
        item.displayInfo()
    }
}

```